Live Coding Python

JSON Robuste, XML Sécurisé, Protobuf

Séance 7 — Sérialisation et Marshalling

Module : Applications Réparties
Pré-requis : Python, JSON, notions API (séance 6)

25 avril 2026

Table des matières

1 Introduction **3**

2 Démo 1 — JSON Robuste avec Validation (`json_robuste.py`) **3**
2.1 Objectif pédagogique 3
2.2 Concepts abordés 3
2.3 Code source 3
2.4 Points pédagogiques 5
2.5 Résultat d'exécution 6

3 Démo 2 — XML Parsing Sécurisé (`xml_securise.py`) **7**
3.1 Objectif pédagogique 7
3.2 Concepts abordés 7
3.3 Code source 7
3.4 Points pédagogiques 8
3.5 Résultat d'exécution 9

4 Démo 3 — Encodage Binaire de type Protobuf (`protobuf_demo.py`) **10**
4.1 Objectif pédagogique 10
4.2 Concepts abordés 10
4.3 Code source 10
4.4 Points pédagogiques 12
4.5 Résultat d'exécution 13

5 Synthèse Comparative **14**
5.1 Pièges fréquents à éviter 14

1 Introduction

Cette section présente trois extraits de code Python illustrant les concepts fondamentaux de la séance 7 : sérialisation JSON robuste avec validation, parsing XML sécurisé (défense contre XXE), et encodage binaire de type Protocol Buffers.

N°	Fichier	Concept clé	Bibliothèque
1	`json_robuste.py`	Dataclass, Validation, Fail Closed	`json`, `dataclasses`
2	`xml_securise.py`	Parsing défensif, anti-XXE	`xml.etree`
3	`protobuf_demo.py`	Encodage binaire, comparaison JSON	`struct`

Table 1 – Vue d'ensemble des trois démonstrations

2 Démo 1 — JSON Robuste avec Validation (`json_robuste.py`

2.1 Objectif pédagogique

Implémenter un pipeline complet de sérialisation/désérialisation JSON en Python pur, intégrant les bonnes pratiques professionnelles : **dataclass** pour le modèle, **exclusion de champs sensibles** lors de la sérialisation, et **validation exhaustive** avec approche *fail closed* lors de la désérialisation.

2.2 Concepts abordés

- **Dataclass Python** : modélisation structurée des entités.
- **Exclusion de champs sensibles** : le champ `_internal_score` n'est jamais sérialisé.
- **Validation stricte** : présence, types, longueurs, allowlist.
- **Fail closed** : toute anomalie entraîne un rejet total.
- **Erreur générique côté client** : `"Payload invalide"`, détail dans les logs.

2.3 Code source

```python
import json
import logging
from dataclasses import dataclass, field, asdict
from typing import List

logger = logging.getLogger(__name__)

@dataclass
class Document:
    id: int
    title: str
    author: str
    tags: List[str] = field(default_factory=list)
    classification: str = "internal"
```

```python
    _internal_score: float = field(
        default=0.0, repr=False
    )

EXCLUDED_FIELDS = {"_internal_score"}

def serialize_document(doc: Document) -> str:
    """Serialise un Document en JSON, sans champs
    sensibles."""
    data = {k: v for k, v in asdict(doc).items()
            if k not in EXCLUDED_FIELDS}
    return json.dumps(data, ensure_ascii=False)

ALLOWED_CLASSIFICATIONS = {
    "public", "internal", "confidential", "secret"
}
MAX_TITLE_LEN = 200
MAX_TAGS = 20

def deserialize_document(raw: str) -> Document:
    """Deserialise et valide un JSON en Document."""
    try:
        data = json.loads(raw)
    except json.JSONDecodeError as e:
        logger.warning("JSON invalide : %s", e)
        raise ValueError("Payload invalide")

    if not isinstance(data, dict):
        raise ValueError("Payload invalide")

    errors = []

    for f in ("id", "title", "author"):
        if f not in data:
            errors.append(
                f"Champ obligatoire manquant : {f}"
            )

    if "id" in data and not isinstance(data["id"], int):
        errors.append("'id' doit etre un entier")

    if "title" in data:
        if not isinstance(data["title"], str):
            errors.append("'title' doit etre une chaine")
        elif len(data["title"]) > MAX_TITLE_LEN:
            errors.append("Titre trop long")

    if ("author" in data
            and not isinstance(data["author"], str)):
        errors.append("'author' doit etre une chaine")

```

```python
    tags = data.get("tags", [])
    if not isinstance(tags, list):
        errors.append("'tags' doit etre une liste")
    elif len(tags) > MAX_TAGS:
        errors.append("Trop de tags")

    classification = data.get(
        "classification", "internal"
    )
    if classification not in ALLOWED_CLASSIFICATIONS:
        errors.append("Classification non autorisee")

    if errors:
        logger.warning("Validation echouee : %s", errors)
        raise ValueError("Payload invalide")

    return Document(
        id=data["id"],
        title=data["title"].strip(),
        author=data["author"].strip(),
        tags=tags,
        classification=classification,
    )


if __name__ == "__main__":
    doc = Document(
        id=1, title="Rapport Q1", author="Alice",
        tags=["finance"], _internal_score=9.5
    )

    json_out = serialize_document(doc)
    print("Serialise :", json_out)

    doc2 = deserialize_document(json_out)
    print("Deserialise :", doc2)

    try:
        deserialize_document(
            '{"id": "abc", "title": 123}'
        )
    except ValueError as e:
        print("Rejete :", e)
```

Listing 1 – json_robuste.py — Serialisation/Deserialisation JSON avec validation

2.4 Points pédagogiques

— Le champ `_internal_score` est exclu de la sérialisation via un filtrage explicite → **pas de fuite de données sensibles**.

— La validation est **exhaustive** : chaque champ est vérifié en présence, type, longueur

et valeur.

— L'erreur côté client est **générique** (`"Payload invalide"`) ; les détails techniques sont envoyés uniquement dans les **logs** côté serveur.

— L'approche **fail closed** : dès qu'une erreur est détectée, le payload entier est rejeté.

A retenir

Validation ≠ confiance. Même un JSON syntaxiquement correct et bien typé peut contenir des valeurs malveillantes. La validation est un filtre nécessaire mais non suffisant : il faut aussi vérifier l'origine (auth) et les autorisations.

2.5 Résultat d'exécution

```
PS C:\Users\imada\Downloads> python json_robuste.py
============================================================
  Demo 1 — JSON Robuste (Seance 7)
============================================================

--- Test 1 : Serialisation ---
  Objet   : Document(id=1, title='Rapport Q1', author='Alice', tags=['finance'], classificatio
n='internal')
  JSON    : {"id": 1, "title": "Rapport Q1", "author": "Alice", "tags": ["finance"], "classifi
cation": "internal"}
  '_internal_score' present ? False

--- Test 2 : Deserialisation valide ---
  Reconstruit : Document(id=1, title='Rapport Q1', author='Alice', tags=['finance'], classific
ation='internal')

--- Test 3 : Classification 'secret' ---
  Reconstruit : Document(id=2, title='Plan defense', author='Bob', tags=[], classification='se
cret')

--- Test 4 : id invalide (string) ---
WARNING: Validation echouee : ["'id' doit etre un entier"]
  Rejete : Payload invalide

--- Test 5 : 'author' manquant ---
WARNING: Validation echouee : ['Champ obligatoire manquant : author']
  Rejete : Payload invalide

--- Test 6 : Classification 'top_secret' ---
WARNING: Validation echouee : ['Classification non autorisee']
  Rejete : Payload invalide

--- Test 7 : JSON malformed ---
WARNING: JSON invalide : Expecting property name enclosed in double quotes: line 1 column 2 (c
```

FIGURE 1 – Exécution de `json_robuste.py` : sérialisation (sans champ sensible), désérialisation valide, et rejet de payloads invalides.

3 Démo 2 — XML Parsing Sécurisé (`xml_securise.py`)

3.1 Objectif pédagogique

Montrer comment parser un document XML de manière **défensive** en Python, en implémentant les contrôles de sécurité essentiels : limitation de taille, détection de DOCTYPE/ENTITY, extraction par allowlist de champs.

3.2 Concepts abordés

— **XXE (XML External Entity)** : une entité XML peut référencer des fichiers locaux ou des URLs → exfiltration de données.

— **Bombe XML** : entités imbriquées exponentielles → déni de service.

— **Défense en profondeur** : limiter la taille AVANT le parsing, puis rejeter les DOCTYPE/ENTITY.

— **Extraction par allowlist** : ne lire que les champs explicitement attendus.

3.3 Code source

```python
import xml.etree.ElementTree as ET
import logging

logger = logging.getLogger(__name__)

MAX_XML_SIZE = 1_000_000  # 1 Mo maximum
EXPECTED_FIELDS = {"id", "title", "author", "date"}


def parse_document_xml(raw_xml: str) -> dict:
    """Parse un XML de maniere defensive."""

    # Controle 1 : taille maximale AVANT parsing
    if len(raw_xml) > MAX_XML_SIZE:
        logger.warning(
            "XML trop volumineux : %d octets",
            len(raw_xml)
        )
        raise ValueError("Payload trop volumineux")

    # Controle 2 : rejet DOCTYPE/ENTITY
    if ("<!DOCTYPE" in raw_xml.upper()
            or "<!ENTITY" in raw_xml.upper()):
        logger.warning(
            "DOCTYPE/ENTITY detecte  rejete"
        )
        raise ValueError("Payload invalide")

    # Controle 3 : parsing
    try:
        root = ET.fromstring(raw_xml)
```

```python
    except ET.ParseError as e:
        logger.warning("XML mal forme : %s", e)
        raise ValueError("Payload invalide")

    # Controle 4 : extraction par allowlist
    result = {}
    for field_name in EXPECTED_FIELDS:
        elem = root.find(field_name)
        if elem is not None and elem.text:
            result[field_name] = elem.text.strip()

    # Controle 5 : champs obligatoires
    missing = {"id", "title"} - result.keys()
    if missing:
        logger.warning(
            "Champs manquants : %s", missing
        )
        raise ValueError("Payload invalide")

    return result


if __name__ == "__main__":
    xml_valid = """<?xml version="1.0"?>
    <document>
        <id>42</id>
        <title>Rapport annuel</title>
        <author>Bob</author>
        <date>2026-01-15</date>
    </document>"""

    result = parse_document_xml(xml_valid)
    print("Parse :", result)
```

Listing 2 – xml_securise.py — Parsing XML defensif

3.4 Points pédagogiques

— Le contrôle de **taille** est effectué **avant** tout parsing : un XML de 10 Ko peut consommer des Go de RAM si des entités imbriquées sont présentes.

— La détection de `<!DOCTYPE` et `<!ENTITY` est une **défense en profondeur** : même si le parseur ne résout pas les entités, on préfère rejeter préventivement.

— L'extraction par **allowlist** (`EXPECTED_FIELDS`) garantit que les champs inattendus sont ignorés — y compris des champs potentiellement injectés.

— En production avec entrées non fiables, il faut utiliser `defusedxml` qui désactive automatiquement les fonctionnalités dangereuses.

Point cybersécurité

XML + données non fiables = danger. Les parseurs XML standard résolvent les entités externes par défaut. C'est l'une des vulnérabilités les plus courantes (OWASP Top 10 : XXE). Préférer JSON pour les nouvelles API, et utiliser `defusedxml` lorsque XML est imposé.

3.5 Résultat d'exécution

```
PS C:\Users\imada\Downloads> python xml_securise.py
============================================================
  Demo 2 — XML Parsing Securise (Seance 7)
============================================================

--- Test 1 : XML valide ---
  Parse : {'id': '42', 'author': 'Bob', 'date': '2026-01-15', 'title': 'Rapport annuel'}

--- Test 2 : Champ inconnu 'secret_field' (ignore) ---
  Parse : {'id': '43', 'author': 'Alice', 'date': '2026-03-01', 'title': 'Plan strategique'}
  'secret_field' present ? False

--- Test 3 : DOCTYPE detecte (protection XXE) ---
WARNING: DOCTYPE/ENTITY detecte dans XML — rejete
  Rejete : Payload invalide

--- Test 4 : XML mal forme ---
WARNING: XML mal forme : no element found: line 1, column 31
  Rejete : Payload invalide

--- Test 5 : 'title' manquant ---
WARNING: Champs manquants dans XML : {'title'}
  Rejete : Payload invalide

--- Test 6 : Payload trop gros (simule) ---
WARNING: XML trop volumineux : 1000022 octets
  Rejete : Payload trop volumineux

============================================================
  Tous les tests termines.
============================================================
```

FIGURE 2 – Exécution de `xml_securise.py` : parsing valide, rejet de DOCTYPE, XML mal formé et champs manquants.

4 Démo 3 — Encodage Binaire de type Protobuf (`protobuf_demo.py`)

4.1 Objectif pédagogique

Illustrer le principe fondamental de Protocol Buffers (Protobuf) en simulant un encodage/décodage binaire avec `struct`, puis comparer la taille du payload avec JSON sur les mêmes données.

4.2 Concepts abordés

- **Encodage binaire** : représentation compacte des données (entiers en 4 octets, strings avec préfixe de longueur).
- **Schéma formel** : la structure est définie par le code d'encodage/décodage (en vrai Protobuf : le fichier `.proto`).
- **Comparaison JSON vs binaire** : mesure concrète du gain de taille.
- **Compatibilité** : ajout de champs optionnels sans casser les anciens lecteurs.

4.3 Code source

```python
import json
import struct

def encode_document(doc_id, title, author,
                    tags, classification=""):
    """Encode en format binaire compact."""
    parts = []

    # Champ 1 : id (int32, 4 octets)
    parts.append(struct.pack(">I", doc_id))

    # Champ 2 : title (prefixe longueur)
    title_b = title.encode("utf-8")
    parts.append(struct.pack(">H", len(title_b)))
    parts.append(title_b)

    # Champ 3 : author
    author_b = author.encode("utf-8")
    parts.append(struct.pack(">H", len(author_b)))
    parts.append(author_b)

    # Champ 4 : tags (repeated string)
    parts.append(struct.pack(">H", len(tags)))
    for tag in tags:
        tag_b = tag.encode("utf-8")
        parts.append(struct.pack(">H", len(tag_b)))
        parts.append(tag_b)

    # Champ 5 : classification
```

```python
    class_b = classification.encode("utf-8")
    parts.append(struct.pack(">H", len(class_b)))
    parts.append(class_b)

    return b"".join(parts)


def decode_document(data):
    """Decode depuis le format binaire."""
    offset = 0

    doc_id = struct.unpack_from(">I", data, offset)[0]
    offset += 4

    title_len = struct.unpack_from(
        ">H", data, offset
    )[0]
    offset += 2
    title = data[offset:offset+title_len].decode()
    offset += title_len

    author_len = struct.unpack_from(
        ">H", data, offset
    )[0]
    offset += 2
    author = data[offset:offset+author_len].decode()
    offset += author_len

    tags_count = struct.unpack_from(
        ">H", data, offset
    )[0]
    offset += 2
    tags = []
    for _ in range(tags_count):
        tl = struct.unpack_from(">H", data, offset)[0]
        offset += 2
        tags.append(data[offset:offset+tl].decode())
        offset += tl

    cl = struct.unpack_from(">H", data, offset)[0]
    offset += 2
    classification = data[offset:offset+cl].decode()
    offset += cl

    return {
        "id": doc_id, "title": title,
        "author": author, "tags": tags,
        "classification": classification or "(defaut)"
    }


```

```python
if __name__ == "__main__":
    binary = encode_document(
        42, "Rapport Q1", "Alice",
        ["finance", "interne"], "confidential"
    )
    print(f"Binaire : {len(binary)} octets")

    json_d = json.dumps({
        "id": 42, "title": "Rapport Q1",
        "author": "Alice",
        "tags": ["finance", "interne"],
        "classification": "confidential"
    }).encode("utf-8")
    print(f"JSON    : {len(json_d)} octets")
    print(f"Ratio   : {len(json_d)/len(binary):.1f}x")

    decoded = decode_document(binary)
    print(f"Decode  : {decoded}")
```

Listing 3 – protobuf_demo.py — Encodage binaire + comparaison JSON

4.4 Points pédagogiques

— L'encodage binaire est **2 à 3 fois plus compact** que JSON car il n'utilise pas de clés textuelles, pas de guillemets, pas d'accolades.

— En vrai Protobuf, le fichier `.proto` remplace le code d'encodage/décodage manuel : `protoc` génère automatiquement les fonctions `SerializeToString()` et `ParseFromString()`.

— Le test de compatibilité montre qu'un champ vide (classification absente en v1) n'empêche pas le décodage : c'est le principe du **versioning natif** de Protobuf.

A retenir — Protobuf en 4 points

1. Le schéma (`.proto`) est le **contrat formel**.
2. Les numéros de champ sont **immuables**.
3. Ajouter un champ optionnel est **toujours compatible**.
4. Le binaire est **2–10× plus compact** que JSON.

4.5 Résultat d'exécution

```
PS C:\Users\imada\Downloads> python protobuf_demo.py
============================================================
  Demo 3 — Protobuf Conceptuel (Seance 7)
============================================================

--- Encodage binaire (simulation Protobuf) ---
  Taille binaire : 57 octets
  Hex (extrait)  : 0000002a000a526170706f72742051310005416c...

  Taille JSON    : 118 octets

  Ratio : JSON est 2.1x plus gros que le binaire

--- Decodage binaire ---
  id                   = 42
  title                = Rapport Q1
  author               = Alice
  tags                 = ['finance', 'interne']
  classification       = confidential

--- Test compatibilite : v1 sans classification ---
  Classification : '(defaut)'
  (vide = ignoree par v1, compatible)

--- Tableau comparatif ---
  Critere                              JSON    Binaire
  ----------------------------- ---------- ----------
  Taille (octets)                       118         57
  Lisibilite                            Oui        Non
  Validation types                Manuelle     Schema
  Compat. ajout champ           Convention      Natif
  Tooling requis                     Aucun     protoc

============================================================
```

FIGURE 3 – Exécution de `protobuf_demo.py` : encodage binaire, comparaison de taille avec JSON, et décodage.

5 Synthèse Comparative

Exemple	Pattern	Problème résolu	Quand l'utiliser
`json_robuste.py`	Validation stricte, fail closed, exclusion champs	Fuite de données sensibles, payloads malformés	Toute API REST
`xml_securise.py`	Parsing défensif, anti-XXE, allowlist	XXE, bombes XML, DoS	Systèmes legacy, XML imposé
`protobuf_demo.py`	Encodage binaire, schéma formel	Taille payload, versioning fragile	gRPC, microservices, IoT

5.1 Pièges fréquents à éviter

1. **Sérialiser des champs sensibles** (scores internes, tokens, mots de passe) → fuite d'information.
2. **Faire confiance à un JSON syntaxiquement valide** → les données peuvent être malveillantes même si bien formées.
3. **Parser du XML non fiable sans protections** → XXE, exfiltration de fichiers serveur.
4. **Utiliser `pickle` sur des entrées réseau** → exécution de code arbitraire.
5. **Retourner des erreurs détaillées au client** → fuite de structure interne.